

Domain Adaptation with a Myanmar Language Model

Concept & Code Report — Junior Developer Edition
with visual explanations

This report walks through the concepts and code used in class-15.ipynb. It is written for a junior developer who knows Python but is new to language modelling. Each concept is introduced in plain English, illustrated with a diagram, then connected to the exact code in the notebook that implements it. Read it once front-to-back; then keep it next to the notebook as a reference.

1. The learning map — how the concepts connect

Before diving in, here is the chain of ideas. Each concept builds on the one before it. If a concept feels unclear, the one above it is the place to revisit.

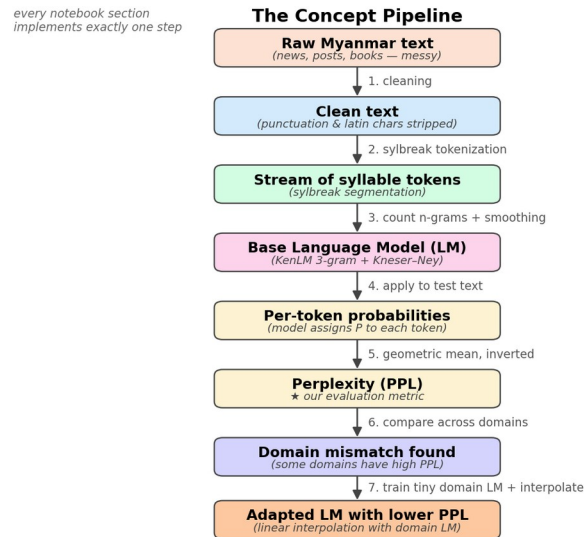


Figure 1. The eight steps of the pipeline. Every section of the notebook implements exactly one step.

Keep this diagram in mind as you read on. The numbered steps on the arrows match the section numbers of the notebook — Section 1 does cleaning, Section 2 does tokenization, and so on.

2. The concepts you must understand

2.1 What is a language model?

A language model is a function that, given some context, tells you how likely each possible next word (or syllable) is. Formally, for a sequence of tokens $w_1 \dots w_n$, an n -gram model assumes each token depends only on the previous $n-1$ tokens:

$$P(w_1 \dots w_n) \approx \prod P(w_i \mid w_{i-n+1} \dots w_{i-1})$$

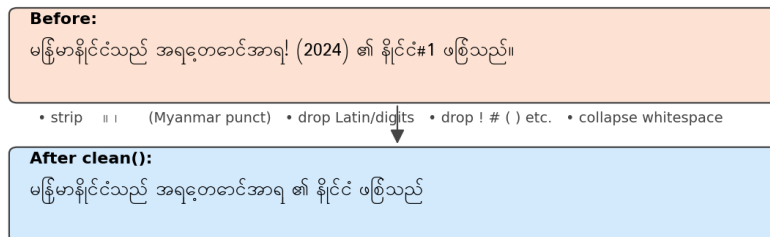
We use a 3-gram (trigram) model: each syllable is predicted from the two syllables before it. KenLM is a fast C++ toolkit that trains and scores n -gram LMs.

→ Leads to: we need to count n -grams in text. To count them, we first need tokens. To get tokens, we need clean text.

2.2 Why we clean the text first

Real Myanmar text has punctuation, Latin letters, digits, and occasionally HTML residue. None of that is a Myanmar syllable, and feeding it into the model would create noise tokens. So before tokenizing, we strip Myanmar punctuation (။ ပုဒ်မ၊ ၊ ပုဒ်စီ), Latin letters and digits, ASCII / Unicode punctuation (! ? , . — “ ” ...) and collapse multiple spaces.

Cleaning: what gets stripped



Why? Punctuation and stray characters are not Myanmar syllables — feeding them in would create noise tokens.

Figure 2. Cleaning in one shot. Everything that isn't a Myanmar syllable gets stripped.

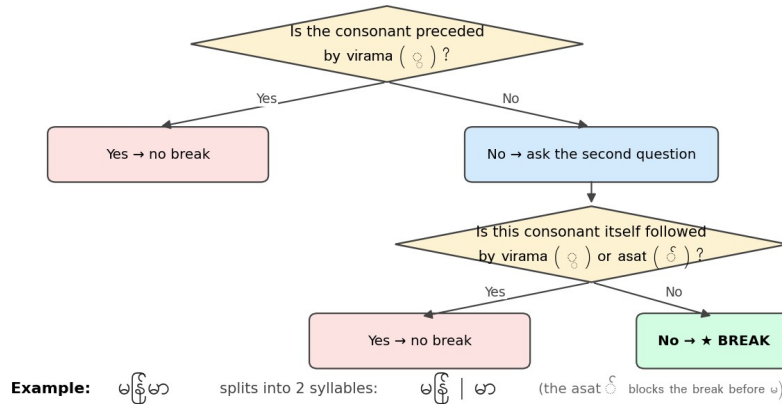
→ Leads to: now we have clean Myanmar text. But Myanmar writes with no spaces between words. How do we split it into tokens?

2.3 Sylbreak — tokenizing Myanmar by syllable

English splits on whitespace. Myanmar does not have whitespace between words, but it has a very regular syllable structure. Ye Kyaw Thu's sylbreak regex segments Myanmar by syllable: a new syllable starts at every Myanmar consonant, EXCEPT when the consonant is preceded by virama (့), or the consonant itself is followed by virama or asat (်).

Sylbreak: where does a new syllable start?

Scan left to right. At each Myanmar consonant, ask two questions:

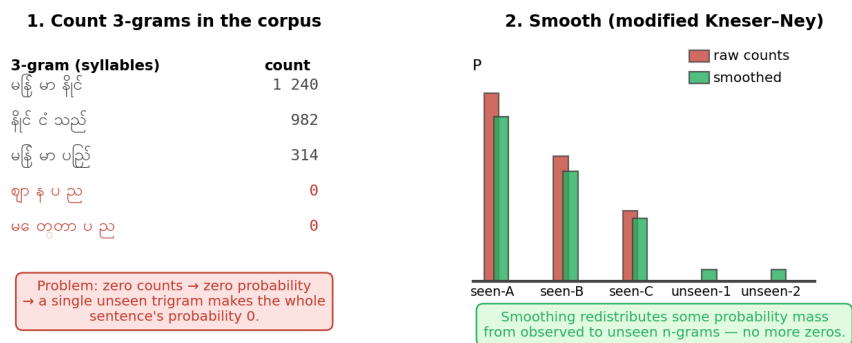
**Figure 3.** The two-question decision tree the sylbreak regex implements at every consonant.

Syllables are a clean, language-appropriate unit for Myanmar LMs. Whether you pick words or syllables matters: perplexity numbers are only comparable when the tokenizer is identical.

→ Leads to: now we have a stream of syllable tokens. We can finally count n-grams and build a language model.

2.4 Smoothing — why raw counts aren't enough

If you only use observed n-gram counts, any unseen n-gram gets probability zero. That breaks the whole model — a single unseen trigram in test text makes the whole sentence's probability zero. Smoothing redistributes some probability mass from observed n-grams to unseen ones.

From raw counts to smoothed probabilities**Figure 4.** Left: raw counts have zero rows. Right: after smoothing, every n-gram has some probability.

Modified Kneser–Ney smoothing (Chen & Goodman, 1998) is the standard for n-gram LMs. KenLM's Implz uses it by default. You don't need to understand the math to use it, but you should know it exists

and that every probability you read out of the model has been smoothed. The flag `--discount_fallback` is only needed for tiny corpora (like ours); on real data, drop it.

→ Leads to: we now have a working LM. How do we measure if it's good?

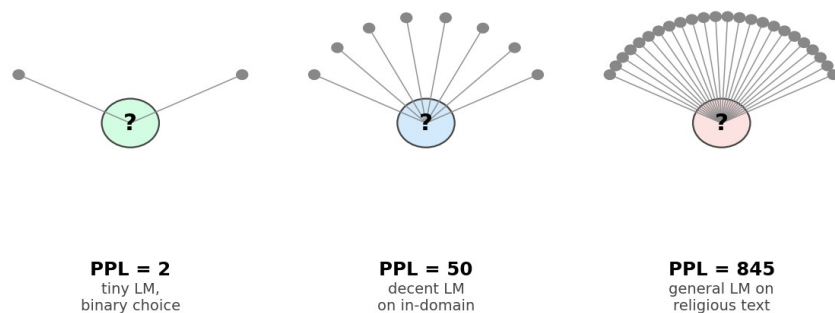
2.5 Perplexity — the evaluation metric

Perplexity (PPL) is the LM's average per-token uncertainty, exponentiated. Intuitively, a PPL of 50 means the model is "about as confused as if it had 50 equally-likely choices per token."

$$PPL(text) = 2^{(-1/N \cdot \sum \log_2 P(w_i | context))}$$

Perplexity = the model's average branching factor

Intuition: "If the model is this confused, how many equally-likely options does it feel like it has at each step?"



Lower PPL = better fit. PPLs are ONLY comparable when tokenizer and test length match.

Figure 5. The branching-factor intuition. A higher PPL = more options the model is choosing between at each step.

Two practical rules: Lower PPL = better fit. And PPLs are only comparable between models that use the SAME tokenizer and the SAME test-token count. That's why our test sets are all exactly 10 sentences × 20 syllables = 200 tokens.

→ Leads to: we measure PPL on three different domains. We discover the model is great in-domain but bad out-of-domain.

2.6 Domain mismatch — why one LM doesn't fit all text

A language model is a frozen snapshot of the vocabulary and patterns in its training data. The further a test text drifts from that, the higher the PPL. For Myanmar:

- Religious gāthā uses Pali loanwords (ဘိလ, မေတ္တ, နိဗ္ဗာန, ဈာန်) absent in general prose.
- Legal Myanmar uses formal terms (ပုဒ်မ, တရားရုံး, ဥပဒေစာ) and heavy nominal syntax.
- Facebook Myanmar uses colloquial particles (ဟေ့, မှီ, နေ), slang and abbreviations.

Why some domains break a general LM

Each domain's vocabulary only partly overlaps with the base LM. The non-overlapping part is what hurts PPL.

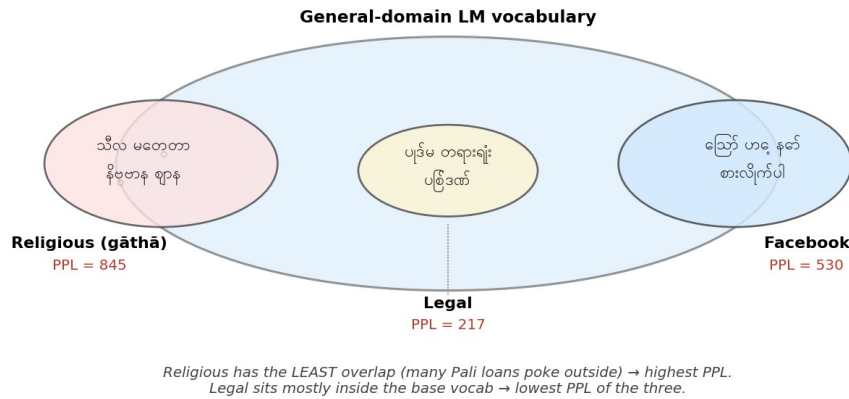


Figure 6. Vocabulary overlap drives PPL. The bigger the part of a domain that pokes outside the base vocab, the worse the PPL.

→ Leads to: we need a way to improve the LM on each domain without retraining from scratch.

2.7 Domain adaptation by linear interpolation

We keep the big base LM, train a tiny in-domain LM on a small amount of domain text, and blend their probabilities token-by-token:

$$P_{\text{mix}}(w \mid h) = \lambda \cdot P_{\text{dom}}(w \mid h) + (1 - \lambda) \cdot P_{\text{base}}(w \mid h)$$

Linear interpolation: blend two LMs at the probability level

$$P_{\text{mix}}(w \mid h) = \lambda \cdot P_{\text{dom}}(w \mid h) + (1 - \lambda) \cdot P_{\text{base}}(w \mid h)$$

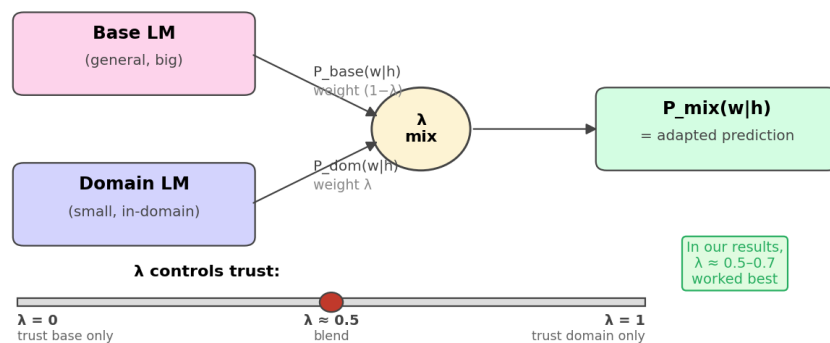


Figure 7. The two LMs are scored independently, then blended at every token with weight λ .

λ is a number between 0 and 1 that controls trust: $\lambda=0$ means trust the base LM only, $\lambda=1$ means trust the tiny domain LM only. The sweet spot (often around 0.5–0.7 in our experiments) blends both. We sweep $\lambda \in \{0.1, \dots, 0.7\}$ and pick the value that minimises PPL.

Linear interpolation is the classical baseline for adaptation. It is fast, easy to implement, and surprisingly effective even with very little in-domain text.

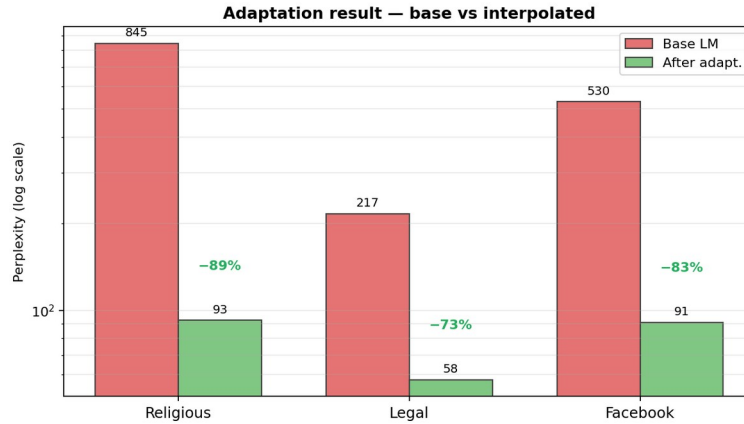


Figure 8. End result: even ~100–200 syllables of in-domain text cuts PPL by 70–90%.

→ End of concept chain. From here we are just running and measuring.

3. Code walkthrough — section by section

The notebook is divided into eight numbered sections. Each section implements exactly one step of the pipeline described in Section 1 of this report.

3.0 Setup

Imports the libraries, sets up the paths, reads KENLM_BIN from the environment, and asserts that the Implz binary is reachable.

```
KENLM_BIN = os.environ.get("KENLM_BIN", "/home/claude/kenlm/build/bin")
assert (Path(KENLM_BIN)/"implz").exists(), f"implz not found at {KENLM_BIN}"
```

If this cell fails on your machine, you either have not built the KenLM binaries or have not set KENLM_BIN before launching Jupyter. See the project README for the setup steps.

3.1 Section 1 — Corpus inspection

Reads data/train/general_corpus.txt, prints character/line counts, and shows a sample. Nothing tricky here — just a sanity check that the input file is present and not empty.

3.2 Section 2 — Cleaning and tokenization

Two helper functions implement the cleaning + sylbreak concepts:

```
def clean(text: str) -> str:
    text = re.sub(r"[A-Za-z0-9]+", " ", text)
    text = re.sub(rf"[{re.escape(PUNCT_TO_STRIP)}]", " ", text)
    return re.sub(r"\s+", " ", text).strip()
```

```
def sylbreak(text: str, sep: str = " ") -> str:
    return break_pattern.sub(rf"{sep}\1", text).strip()
```

tokenize_file streams a file line-by-line, runs clean then sylbreak on each line, and writes a syllable-tokenized version. Run on the training corpus, it produces general_corpus.syl.txt.

Why stream line-by-line instead of loading the whole file? On real multi-GB corpora, loading everything into RAM is wasteful. The function is already written to scale up.

3.3 Section 3 — Train the base LM

We call the Implz binary as a subprocess: stdin is the tokenized corpus, stdout is the ARPA model file.

```
cmd = [f"{KENLM_BIN}/implz", "-o", "3", "--discount_fallback", "-S", "30%"]
with open(train_syl) as fin, open(arpa, "w") as fout:
    r = subprocess.run(cmd, stdin=fin, stdout=fout, stderr=subprocess.PIPE)
```

Flag meanings:

- `-o 3` — train a trigram model.
- `-S 30%` — use up to 30% of available RAM during sorting/counting.
- `--discount_fallback` — required for tiny corpora; drop on real data.

Then build_binary converts the human-readable ARPA into a memory-mapped .bin for fast loading. Finally kenlm.Model(...) loads the binary into Python so we can score with it.

3.4 Section 4 — Domain test sets

Three raw test files (religious_raw.txt, legal_raw.txt, facebook_raw.txt) are tokenized with the SAME sylbreak code, then trimmed to exactly 200 syllable tokens each (10 sentences × 20 syllables).

Why exactly the same length? Perplexity averages logprob across tokens. Different test lengths would give numbers that are not directly comparable.

3.5 Section 5 — Perplexity per domain

One line does the actual measurement:

```
base_ppls[dom] = base_lm.perplexity(text)
```

KenLM's perplexity() adds <s> / </s> markers and averages logprob over tokens. We also score a slice of the training data as a sanity floor — the in-domain PPL should always be the lowest of the four bars.

The bar chart is on a log scale because the religious PPL is roughly 10× larger than the legal one. A linear y-axis would squash the smaller bars into invisibility.

3.6 Section 6 — Adaptation strategies (brainstorm)

A markdown table of options:

- **(a) Mix domain data into training** — cheap, but dilutes the general LM and forces full retraining.
- **(b) Linear interpolation** — what we use. Tunable per domain.

- **(c) Vocabulary expansion** — add domain syllables to the LM vocabulary.
- **(d) Class-based LM** — group rare words into classes.
- **(e) Fine-tune a neural LM** — the modern default, but outside KenLM.

3.7 Section 7 — Adaptation experiment

For each domain, `train_domain_lm` holds out the first 200 syllables (the test set) and trains a tiny 2-gram LM on the remaining in-domain syllables. `interp_ppl` then walks through the test text token by token:

```
for (bp, _, _), (dp, _, _) in zip(b, d):
    p = lam * (10 ** dp) + (1 - lam) * (10 ** bp)
    total_logp += math.log10(p); total_toks += 1
return 10 ** (-total_logp / total_toks)
```

Reading the code:

- `full_scores()` returns log10 probabilities — that's why we do `10 ** dp` to get back to linear probability before mixing.
- After mixing, we go back to log10 to accumulate (avoids floating-point underflow on long texts).
- Final PPL is $10^{(-\text{average_logprob})}$ — the standard PPL formula adapted to base 10.

The sweep over λ values picks the best mix per domain. Results are plotted in `lambda_sweep.png` and `before_after.png` in the outputs folder.

4. Key takeaways for a junior dev

If you remember only five things from this report:

- **1. A language model is just a probability distribution over token sequences.** Don't be intimidated — the math is plumbing around counting and smoothing.
- **2. Tokenization is part of the model.** Change the tokenizer and you change the language model. PPLs only compare under identical tokenization.
- **3. Perplexity is your friend, but it is sensitive.** Same tokenizer, same length, same vocab — only then can you compare two PPL numbers.
- **4. Domain mismatch is the single biggest reason LMs underperform in practice.** Always test on data that looks like production traffic.
- **5. You can fix a lot of domain mismatch with very little in-domain data.** Linear interpolation with even 100 syllables of domain text cuts PPL by 70–90%. Bigger fancier techniques are not always needed.

5. Glossary

n-gram — A contiguous sequence of n tokens. A 3-gram of syllables is three consecutive syllables.

ARPA file — Plain-text format for n -gram LMs. Human-readable but slow to load.

Binary LM (.bin) — Memory-mapped compiled form of an ARPA file. Loads instantly, scores fast.

Smoothing — Redistribution of probability mass so unseen n-grams have non-zero probability.

Kneser-Ney — A specific (and effective) smoothing recipe. The default in Implz.

Perplexity (PPL) — Average per-token uncertainty, exponentiated. Lower is better.

OOV (out-of-vocabulary) — A token in the test set that was never seen during training.

Domain — A coherent subset of text — religious verse, legal contracts, social posts, etc.

Linear interpolation — Adaptation method: $P_{\text{mix}} = \lambda \cdot P_{\text{dom}} + (1-\lambda) \cdot P_{\text{base}}$.

Sylbreak — A regex that segments Myanmar text by syllable boundary.

Virama (့) — Myanmar combining mark that stacks consonants. Prevents a syllable break.

Asat (့း) — Myanmar combining mark that marks the end of a consonant cluster. Also prevents a syllable break before the next consonant.

6. Where to go next

- Jurafsky & Martin, Speech and Language Processing, Chapter 3 (n-gram LMs) — the canonical textbook chapter, free online.
- Chen & Goodman 1998, 'An Empirical Study of Smoothing Techniques for Language Modeling' — the paper behind modified Kneser-Ney.
- Kenneth Heafield's KenLM paper (2011) and the source at github.com/kpu/kenlm.
- For modern neural LMs, the natural next step after this notebook is a small Transformer trained on the same Myanmar corpus — compare its PPL to KenLM's.